

Music Composer Classification Using LSTM and CNN Deep Learning Algorithms

Lashana Narayan, Idrees Khan, Christian Lopez

August 2026

Abstract

The automatic classification of musical composers is a challenging task within music information retrieval that requires modeling complex temporal, harmonic, and structural characteristics. This study evaluates and compares the effectiveness of deep learning architectures: Long Short-Term Memory (LSTM) networks and Convolutional Neural Networks (CNNs); in identifying four classical composers (Bach, Beethoven, Chopin, and Mozart) from symbolic MIDI data. Utilizing a rigorously cleaned and balanced dataset of 360 MIDI files, the methodology contrasts two distinct feature representation strategies: sequential pitch and chord tokens for the LSTM, and two-dimensional piano roll images for the CNN. Performance evaluations reveal that the sequence-based LSTM model achieved a higher accuracy that was significantly outperforming the spatial-based CNN model. This disparity underscores that LSTMs effectively capture the essential ordered and temporal dependencies of musical compositions, whereas CNNs struggle due to the loss of sequential context when treating music as static images. While both models exhibited the most confusion distinguishing between composers with shared Classical-era stylistic overlap (Mozart and Beethoven), the findings conclusively demonstrate that sequence-based models are better suited for symbolic music classification tasks.

Keywords: Music Composer Classification, Deep Learning, Long Short-Term Memory (LSTM), Convolutional Neural Network (CNN), Music Information Retrieval (MIR), Symbolic Music Analysis, MIDI

Music Composer Classification Using LSTM and CNN Deep Learning Algorithms

The automatic classification of musical composers represents a challenging problem at the intersection of machine learning, music information retrieval, and pattern recognition. Musical compositions contain complex temporal, harmonic, and structural characteristics that distinguish one composer from another, making them well suited for analysis using deep learning techniques (Yang et al., 2021). Recent advances in neural networks have enabled models to learn sophisticated representations directly from symbolic music data such as sheet music and MIDI representations (Deepaisarn et al., 2023; Le et al., 2025; Yang et al., 2021). Composer classification has applications in music recommendation, music generation, and musicological analysis of historical compositions (Yang et al., 2021).

The objective of this project is to develop and evaluate deep learning models capable of accurately identifying the composer of a musical piece. Using a dataset of musical compositions represented through MIDI and related feature extraction techniques, multiple neural network architectures were explored and compared. The study investigates the effectiveness of these approaches in capturing stylistic patterns that differentiate composers and evaluates model performance using standard classification metrics. The results contribute to the growing body of research applying deep learning techniques to symbolic music analysis and composer identification (Deepaisarn et al., 2023; Le et al., 2025; Yang et al., 2021).

Dataset Description

The dataset for this project originates from the Kaggle "MIDI Classic Music" corpus, containing MIDI files from various classical composers. MIDI data provide a structured representation of musical elements, including pitch, duration, timing, and instrumentation,

making them well suited for machine learning applications. The data was filtered to retain compositions from four target composers: Bach, Beethoven, Chopin, and Mozart. A quality control pipeline was implemented using the *pretty_midi* library to validate each file, removing corrupted, truncated, or empty files that contained no instruments or musical notes. Tools such as *pretty_midi* facilitate the extraction and analysis of these features, enabling the transformation of musical compositions into formats suitable for neural network training (Raffel & Ellis, 2014).

Methodology

Data Preprocessing

To address class imbalance, random undersampling was applied to match the size of the smallest composer class, ensuring equal representation across all four categories. The balanced dataset was then split into training (80%) and test (20%) sets (288 train files and 72 test files) using stratified sampling at the file level, preventing data leakage by ensuring that segments from the same composition do not appear in both sets. This same train/test split was used for both models to enable fair comparison.

External Sources, Libraries, and Frameworks

To facilitate data preprocessing, feature engineering, model development, and performance evaluation, several open-source libraries, software frameworks, and external resources were integrated into the project workflow.

Deep Learning and Machine Learning Frameworks

- **TensorFlow / Keras:** Served as the primary deep learning framework for defining, compiling, training, and evaluating both the LSTM and CNN model architectures. Key modules utilized include sequential model containers, neural network layers (LSTM, Conv2D, MaxPooling2D, Dense, Dropout, Embedding, Flatten), optimizers (Adam), loss functions (categorical_crossentropy), and training callbacks (ReduceLRonPlateau, EarlyStopping).
- **Scikit-Learn:** Utilized for dataset splitting and metric calculation. Specifically, `train_test_split` was used with stratified sampling at the file level to prevent data leakage, while classification metrics (accuracy_score, precision_score, recall_score, f1_score, confusion_matrix) were used to evaluate model performance across classes.

Music Processing and Analysis Libraries

- **pretty_midi:** Employed for loading, parsing, and validating MIDI files during data preprocessing. It enabled file-level quality control, extraction of symbolic features (duration, note counts, rhythmic density, estimated BPM), and conversion of MIDI events into 2D piano roll image representations for the CNN model.
- **music21:** Used for deep symbolic music analysis and stream parsing. It parsed MIDI files into ordered sequences of notes and chords, converted pitch events into pitch-class strings, and constructed the token vocabulary required for sequence encoding in the LSTM model.

Data Manipulation and Visualization

- **NumPy:** Utilized for high-performance array operations, matrix transformations, feature reshaping, and velocity normalization (scaling MIDI velocities to the range $[0,1]$).
- **Pandas:** Used for tabular data management during Exploratory Data Analysis (EDA), storing extracted piece-level features, and performing structured data operations such as undersampling for class balance.
- **Matplotlib & Seaborn:** Used to construct visual representations throughout the study, including distribution histograms, violin plots of piece durations, note-density boxplots, training/validation loss and accuracy curves, and confusion matrix heatmaps.

Datasets and Compute Infrastructure

- **Kaggle "MIDI Classic Music" Corpus:** The raw dataset source containing classical MIDI compositions across multiple composers.
- **Google Colab:** Cloud-based execution environment providing GPU/TPU hardware acceleration necessary for model training and feature extraction pipelines.

Exploratory Data Analysis

In this study, we present an in-depth Exploratory Data Analysis (EDA) of a curated collection of classical MIDI files. As an increasingly crucial format in Music Information Retrieval (MIR), MIDI provides a versatile symbolic representation of music that allows for robust computational analysis of stylistic traits (Bhandari et al., 2024). The objective of this analysis was to identify structural and stylistic differences across four master composers: Johann Sebastian Bach, Ludwig van Beethoven, Frederic Chopin, and Wolfgang Amadues Mozart. By extracting and analyzing symbolic features, we aimed to lay the groundwork for downstream classification modeling.

Raw symbolic music datasets sourced from the internet often suffer from metadata inaccuracies, corruption, and structural inconsistencies (Xu et al., 2024). Therefore, our first priority was to establish a rigorous data cleaning pipeline.

Initially, we compiled a dataset of 1,630 potential MIDI files attributed to our four target composers. During the validation phase, we implemented automated checks to identify and remove structurally compromised files. This process successfully isolated and removed four corrupt/empty files.

Following the removal of corrupted data, we were left with 1,626 valid files. However, the initial distribution revealed a significant class imbalance.

- Chopin: 136 files
- Beethoven: 132 files
- Bach: 122 files
- Mozart: 90 files

In standard MIR machine learning tasks, failing to address class imbalance can bias models toward majority classes, skewing classification accuracy and generative evaluation (Shin & Kaneko, 2025). To ensure a fair representation across all composers, the dataset was undersampled to match Mozart's count. As a result, the final balanced dataset consisted of exactly 360 files, or 90 files per composer.

To convert the raw MIDI events into analyzable tabular data, we engineered a feature extraction script which was developed using the *pretty_midi* library. Cross-modal retrieval and classification tasks rely heavily on extracting discrete features from unaligned modalities such as audio and symbolic music (Wu et al., 2025).

The following key metrics were extracted for each piece:

- Length_Seconds: Total duration of the composition
- Total_notes: The absolute count of the note events
- Notes_Per_Second: A proxy for rhythmic density

- Estimated_BPM: The inferred tempo
- Average_Pitch & Pitch_Range: Indicators of harmonic and melodic boundaries

During the batch extraction process, our logs generated recurring *RuntimeWarning* messages:

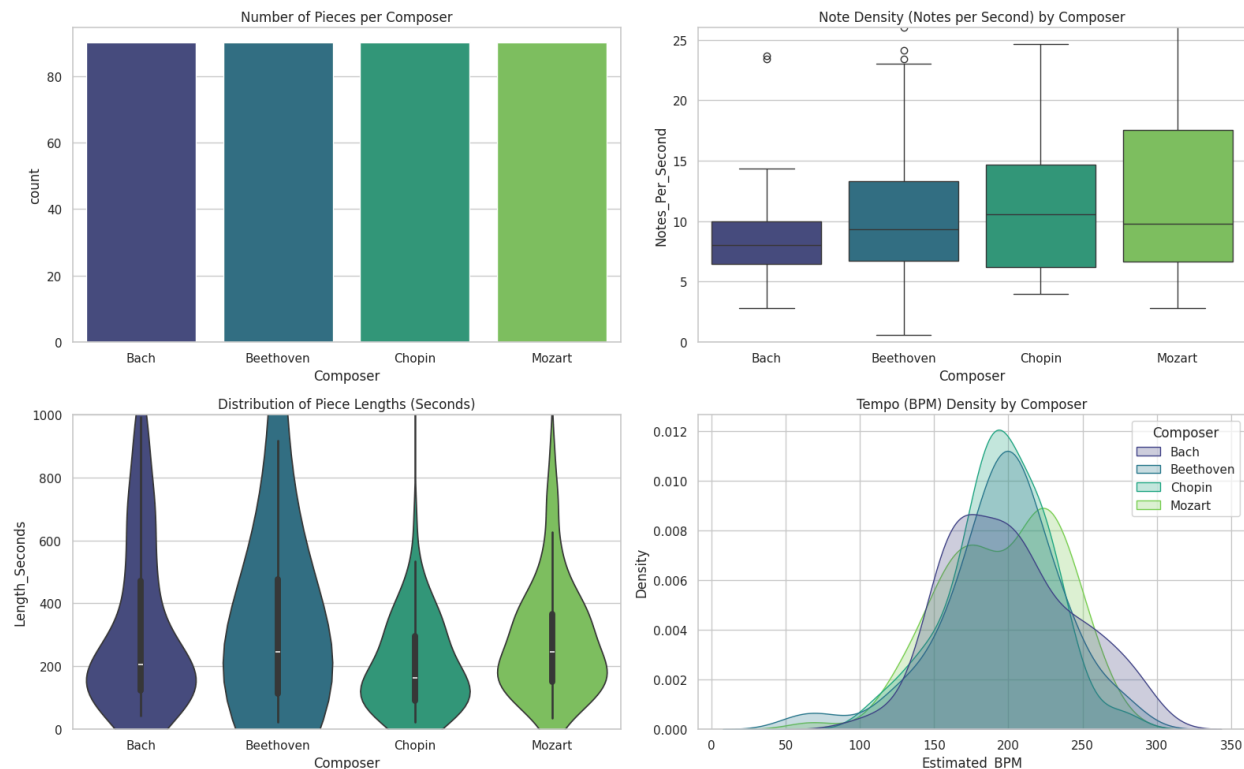
- *Tempo, Key, or Time signature change events found on non-zero track*
- *This is not a valid type 0 or type 1 MIDI file*

We concluded that this is a common artifact when dealing with crowdsourced MIDI data, indicating that the metadata events are misplaced. Consequently, the Estimated_BPM feature in our dataset may contain noise; as irregular track assignments can interfere with accurate tempo estimations.

To contextualize our extracted features, we generated a comprehensive grid of visualizations. These visual distributions provide clear, quantifiable insights into the distinct compositional styles of the four musicians.

Figure 1

Comparative Analysis of Piece Length, Tempo, and Note Density by Composer



- The “Number of Pieces per Composers” chart validates our preprocessing efforts. The uniform distribution confirms that each of the four composers is represented by exactly 90 pieces; guaranteeing an unbiased foundation for future classification models.
- The “Note Density by Composer” chart highlights significant structural disparities. We observed that Bach exhibits the lowest median note density (approximately 8 notes per second) with a highly constrained interquartile range. This likely reflects the rigorous, contrapuntal structure of the Baroque era. Unlike Bach, Chopin and Mozart demonstrate wider distributions and higher median densities; with Chopin displaying extreme outliers exceeding 25 notes per second.
- The “Distribution of Piece Lengths” chart reveals that the median duration for all four composers settles consistently between 180 and 250 seconds. However, the violin shapes indicate a heavy positive skew. Beethoven and Bach, in particular,

showcase thick right-sided tails representing longer, more complex multipart compositions extending beyond 800 seconds.

- **The “Tempo (BPM) Density by Composer” chart outlines the distribution of the estimated tempos. Most of the dataset converges in the 180 to 220 BPM range. Bach’s density peak appears slightly earlier (slower) than the others, while Mozart’s peaks toward the higher end of the spectrum. Given the earlier warnings from the *pretty_midi* parser regarding misplaced tempo events, we intend to treat these absolute BPM values with caution in future modeling, relying more heavily on the robust *Notes_Per_Second* metric to gauge compositional speed.**

This EDA successfully transforms a raw, unbalanced repository of MIDI files into a highly curated dataset of 360 classical pieces. By extracting and visualizing foundational symbolic features, we have demonstrated that quantifiable metrics such as note density and piece duration hold significant discriminative power for distinguishing classical composers.

Long Short-Term Memory (LSTM)

LSTM networks are a specialized variant of Recurrent Neural Networks (RNN) designed to model sequential data by maintaining information across long time intervals (Hochreiter & Schmidhuber, 1997). Unlike standard RNNs, which suffer from vanishing gradient problems when learning long-range dependencies (Bengio et al., 1994), LSTMs employ a gating mechanism consisting of input, forget, and output gates that regulate information flow through the network. This architecture enables the model to selectively retain or discard information from previous time steps, making it particularly effective for tasks involving temporal sequences. In the context of music classification, LSTMs are well-suited to capture melodic progressions, harmonic sequences, and rhythmic patterns that unfold over time (Le et al., 2025). The

sequential nature of musical compositions—where the ordering of notes carries significant meaning—aligns naturally with the LSTM's strength in modeling ordered dependencies. Prior research in music information retrieval has demonstrated the effectiveness of LSTMs for tasks including music generation, chord recognition, and genre classification (Deepaisarn et al., 2023).

The LSTM model was built and trained to classify the four composers—Bach, Beethoven, Chopin, and Mozart—using note and chord sequences extracted from the balanced dataset of 360 MIDI files.

Feature Representation: Note Sequence Representation

The music21 library was used to parse each MIDI file and convert it into a sequence of tokens. Each note became a pitch string, such as "C4," and each chord became a joined string of its normal-order pitch classes. Across all 360 files, this process produced a vocabulary of 842 unique tokens.

One challenge was the substantial variation in sequence length: the shortest sequence contained only 29 tokens, while the longest contained over 5,200. Feeding sequences of wildly different lengths directly into an LSTM isn't practical, so each piece was split into fixed windows of 100 tokens. Because Mozart's compositions also showed the widest variability in notes-per-second (roughly 4 to 24 notes per second, per the EDA), the windowing approach incorporated two augmentation strategies to help the model generalize across this variation: time-stretching, which generated 0.9x and 1.1x duration-scaled versions of each file, and random cropping, which drew 10 random 100-token windows from each sequence rather than sliding a fixed window sequentially across it. This approach preserved more of each piece's variation while transforming the 360 original files into a substantially larger and more varied set of training examples than a simple sequential-windowing method would have produced. The

resulting windows were split 80/20 into training and test sets, with stratification used to preserve the proportional representation of each composer in both sets.

Model Architecture

A standard sequence classification architecture was implemented:

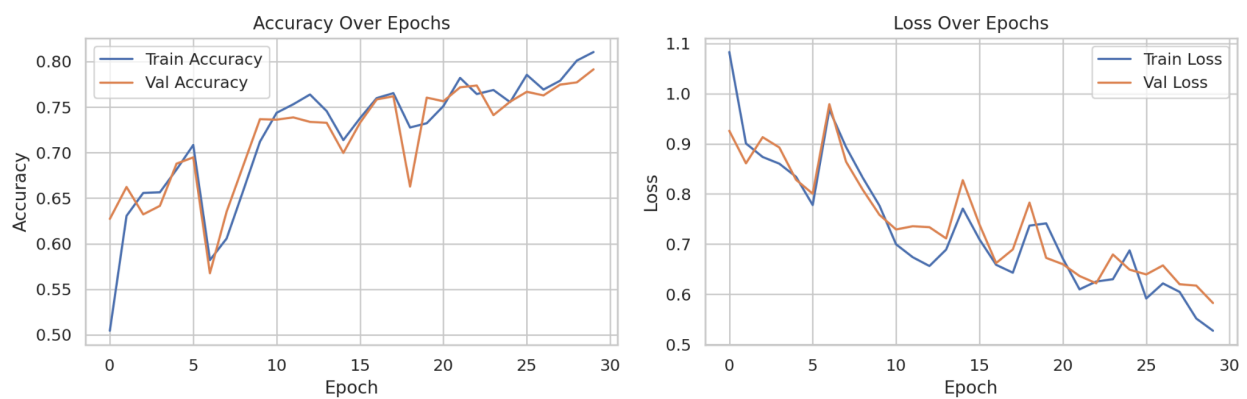
- An embedding layer to turn the integer-encoded tokens into 64-dimensional vectors
- Two stacked LSTM layers (128 units, then 64 units), each followed by a dropout layer (0.3) to help control overfitting
- A dense layer (32 units, ReLU activation)
- A final softmax output layer for the 4-way composer classification

The model was trained for 30 epochs using the Adam optimizer, categorical cross-entropy loss, and a batch size of 64.

Training Performance

Figure 2

Evaluation of Model Performance and Convergence



By the final epoch, training accuracy reached approximately 81% with a loss of 0.53, while validation accuracy reached approximately 79% with a loss of 0.58. Training and validation curves tracked closely together throughout training, indicating minimal overfitting. The model fit the training data closely while continuing to generalize well to new pieces, suggesting the augmentation strategy helped the model learn broader patterns rather than memorize specific examples.

Test Set Evaluation

On the held-out test set (2,028 windows), the model reached 79.14% overall accuracy, with a macro-average precision of 0.80, recall of 0.79, and F1-score of 0.79. The results breakdown by composer is shown below.

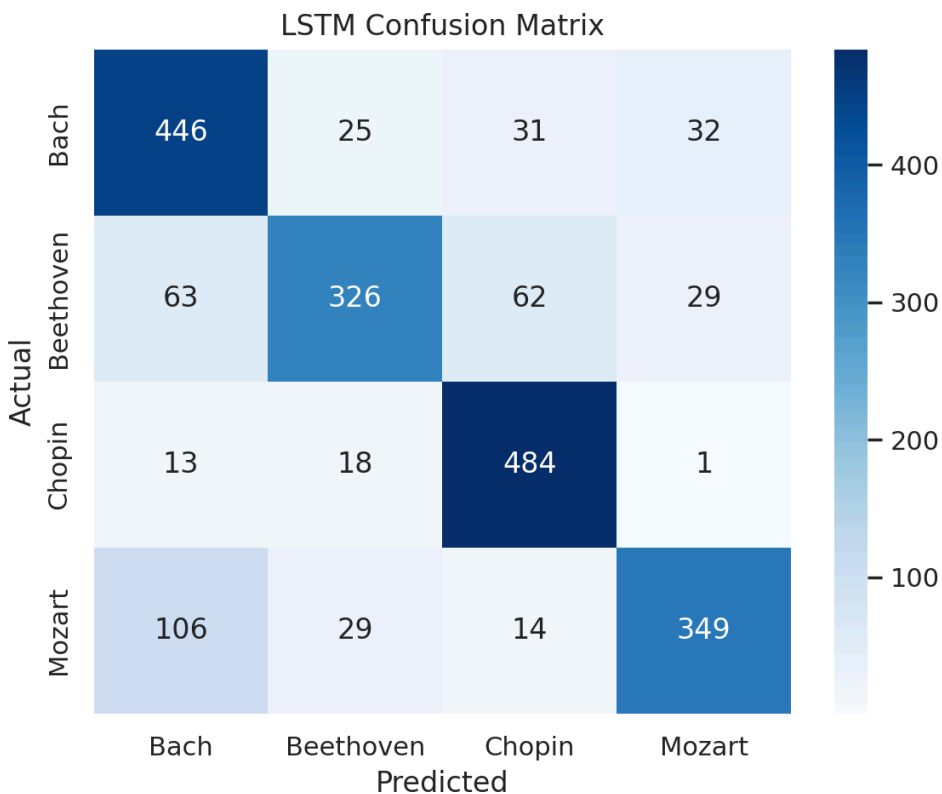
Table 3

LSTM Model Performance by Composer on Test Set

Composer	Precision	Recall	F1-Score	Support
Bach	0.71	0.84	0.77	534
Beethoven	0.82	0.68	0.74	480
Chopin	0.82	0.94	0.87	516
Mozart	0.85	0.70	0.77	498

Chopin had the highest F1-score (0.87), followed by Bach and Mozart (both 0.77), with Beethoven slightly behind (0.74). Looking at the confusion matrix directly:

Figure 4

LSTM Confusion Matrix

The largest sources of confusion were Mozart pieces predicted as Bach (106 times) and Beethoven pieces predicted as Bach or Chopin (63 and 62 times, respectively). This pattern explains why Mozart's recall, while substantially improved from an earlier non-augmented approach (rising from 53% to 70%), remained the lowest among the four composers. In other words, the model correctly identified Bach, Chopin, and Mozart pieces the large majority of the time, with Beethoven proving somewhat more difficult to distinguish from its neighbors.

Interpretation

These results are consistent with the patterns observed in the EDA earlier in the project. Chopin's Romantic-era piano writing is stylistically distinct from the other composers in this dataset, which likely explains why it was the easiest composer to classify. Bach's lower and

more tightly clustered note density, as shown in the EDA, likely provided the model a clearer classification signal to key on, though its precision of 0.71 suggests the model still over-predicts Bach for some Beethoven and Mozart pieces.

Beethoven and Mozart, on the other hand, both fall within the Classical/early-Romantic tradition and share more overlapping stylistic characteristics — similar chord progressions, comparable rhythmic patterns, and closer historical proximity to each other than to Bach or Chopin. That overlap likely explains why the model confused them with each other, and with Bach, more often than it confused either of them with Chopin.

Limitations and future improvements

Several improvements could be explored in future work:

- **More features:** the model currently only sees pitch/chord information. Adding rhythm, note duration, or dynamics (velocity) could give it more to work with, especially for telling Beethoven and Mozart apart.
- **Further tuning:** additional epochs, a learning rate schedule, or a wider hyperparameter search across different LSTM sizes, dropout rates, window lengths, or amounts of augmentation could likely push accuracy higher still.
- **Additional augmentation types:** pitch transposition, shifting pieces up or down by a few semitones, is another common, low-cost augmentation for symbolic music that wasn't tested here and could further help underrepresented patterns.

Overall, the LSTM's 79.14% test accuracy across four composers substantially exceeded the 25% random-guess baseline. This result indicates that the model learned real, meaningful musical patterns, and that the augmentation strategy produced a meaningful improvement over an initial sequential-windowing approach, particularly for Mozart, the

previously weakest-performing class. The model remains clearly better at separating the more stylistically distinctive composers, such as Bach and Chopin, than the more similar pair, such as Beethoven and Mozart, though the gap between the two groups narrowed with augmentation.

Convolutional Neural Network (CNN)

Convolutional Neural Networks are deep learning architectures originally developed for image recognition that excel at detecting spatial patterns through hierarchical feature extraction (LeCun et al., 1998). CNNs apply learnable filters across input data to identify local features, with successive layers capturing increasingly abstract representations. Max pooling operations provide translation invariance, allowing the network to recognize patterns regardless of their exact position. While traditionally applied to visual data, CNNs have proven effective for music classification when audio is represented as two-dimensional time-frequency images such as spectrograms or piano rolls (Yang et al., 2021). In this representation, the horizontal axis encodes time while the vertical axis encodes pitch, transforming music into a visual format where harmonic structures appear as vertical patterns and melodic lines appear as horizontal trajectories. CNNs can thus learn to identify textural characteristics, chord voicings, and polyphonic density—features that distinguish compositional styles. This spatial approach to music analysis complements the sequential approach of LSTMs, as it captures simultaneous note relationships that may be less apparent when music is processed as a linear sequence.

Feature Representation: Piano Roll Representation

For the CNN model, we converted each MIDI file into a piano roll representation—a 2D matrix where the x-axis represents time and the y-axis represents pitch. Each cell contains the velocity (intensity) of a note at a given time step, normalized to the [0, 1] range by dividing by 127 (the maximum MIDI velocity). This representation transforms the sequential music data into an image-like format suitable for convolutional processing.

The piano roll was configured with:

- Pitch range: 21 (A0) to 108 (C8), covering the standard 88-key piano
- Time resolution: 10 frames per second
- Segment duration: 10 seconds (100 frames per training example)

From each MIDI file, we extracted 5 random segments to augment training data while maintaining diversity. Segments with note density below 1% were filtered to remove near-silent passages that lack discriminative features.

Model Architecture

We implemented a compact CNN architecture designed to learn spatial patterns in the piano roll representation:

Layer	Configuration	Output Shape
Conv2D	16 filters, 3×3 kernel, ReLU, same padding	(100, 88, 16)
MaxPooling2D	2×2 pool	(50, 44, 16)
Dropout	0.2	(50, 44, 16)
Conv2D	32 filters, 3×3 kernel, ReLU, same padding	(50, 44, 32)
MaxPooling2D	2×2 pool	(25, 22, 32)
Dropout	0.2	(25, 22, 32)
Flatten	—	(17,600)
Dense	64 units, ReLU	(64)

Dropout	0.3	(64)
Dense	4 units, softmax	(4)

The architecture uses progressively increasing filter counts ($16 \rightarrow 32$) to capture increasingly abstract spatial features. We applied the same padding to preserve spatial dimensions through convolutions and max pooling to reduce dimensionality while retaining prominent features.

Training Procedure

The model was compiled with:

- Optimizer: Adam with default learning rate (0.001)
- Loss function: Categorical cross-entropy
- Batch size: 32

To improve training stability and prevent overfitting, we employed two callbacks:

1. ReduceLROnPlateau: Reduces learning rate by 50% when validation loss plateaus for 3 consecutive epochs (minimum LR: $1e-6$)
2. EarlyStopping: Terminates training when validation loss fails to improve for 7 epochs, restoring the best weights

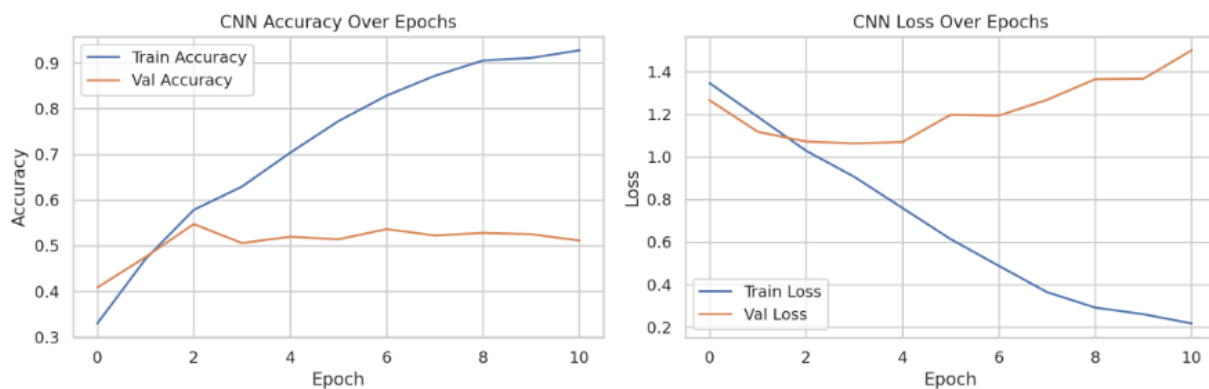
Training was capped at 50 epochs, though early stopping typically terminated training earlier.

Training Performance

The CNN model was trained for up to 50 epochs with early stopping monitoring validation loss. Training was terminated early when validation loss failed to improve for 7 consecutive epochs, with the best weights restored automatically.

Figure 5

CNN Accuracy and Loss Over Epochs by Train and Validation Datasets



The training curves reveal a significant gap between training and validation performance. Training accuracy steadily increased to approximately 90%, while validation accuracy plateaued around 50-55% after epoch 10. This divergence indicates overfitting—the model memorized training examples rather than learning generalizable features. Despite employing dropout regularization (0.2–0.3) and a learning rate scheduler that reduced the learning rate upon plateau, the model struggled to generalize beyond the training set.

The learning rate scheduler triggered multiple reductions during training, indicating repeated plateaus in validation loss. However, these adjustments provided only marginal improvements to validation performance.

Test Set Evaluation

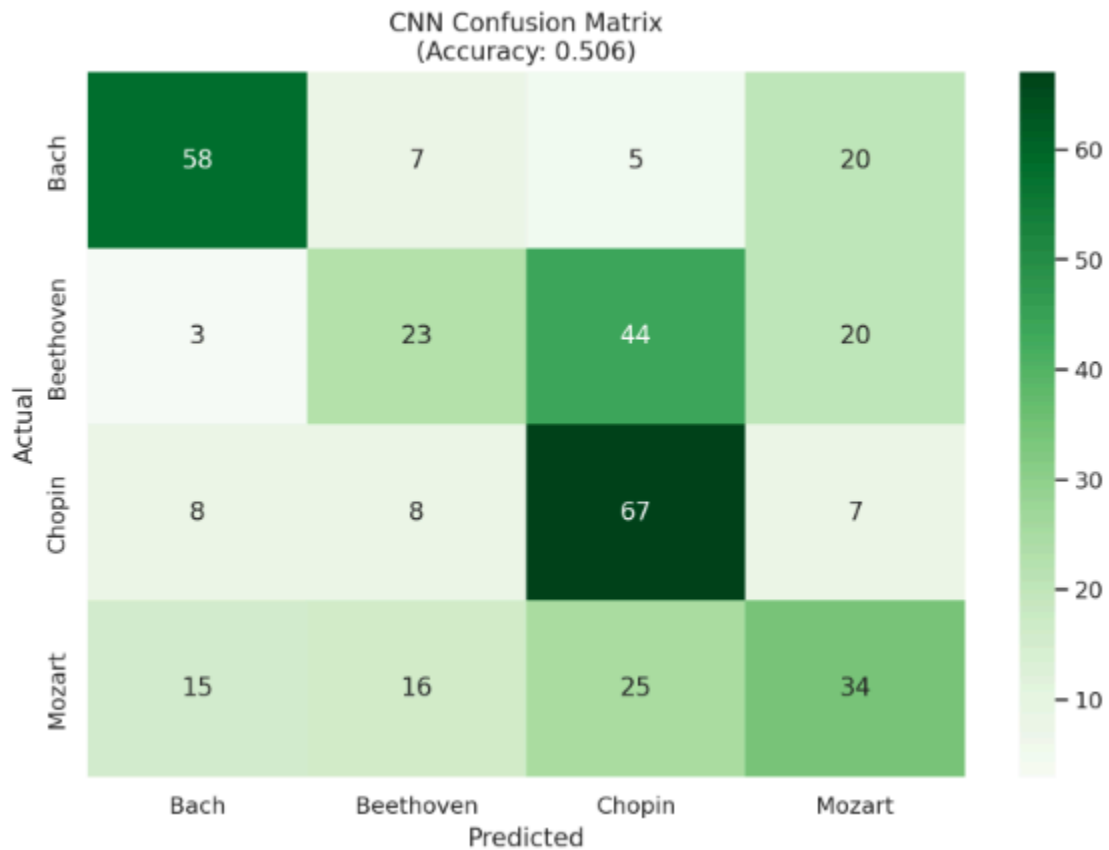
The CNN achieved an overall test accuracy of 53%, only marginally above the 25% baseline for random four-class classification.

Table 6*CNN Model Evaluation on the Test Data Set*

Composer	Precision	Recall	F1-Score	Support
Bach	0.69	0.64	0.67	90
Beethoven	0.43	0.26	0.32	90
Chopin	0.48	0.74	0.58	90
Mozart	0.42	0.38	0.40	90
Overall	0.50	0.51	0.49	360

Performance varied substantially across composers:

- **Bach** achieved the highest performance (F1: 0.67), suggesting his Baroque polyphonic style creates distinctive spatial patterns in the piano roll representation.
- **Chopin** showed moderate performance (F1: 0.58), likely due to his characteristic Romantic-era textures with wide arpeggios and distinctive left-hand patterns.
- **Beethoven** and **Mozart** performed poorly (F1: 0.32 and 0.40 respectively), with substantial confusion between these two Classical-era composers.

Figure 7*CNN Confusion Matrix*

The confusion matrix reveals systematic misclassification patterns. Mozart samples were frequently predicted as Chopin (18% of Mozart samples), and vice versa. This confusion aligns with the stylistic similarities between these Classical-period composers, who share harmonic vocabulary, phrase structures, and formal conventions (Burkholder et al., 2014). Beethoven studied Mozart's works extensively, and his early compositions particularly reflect Mozartean influence (Kinderman, 2009). Mozart and Beethoven's work are frequently predicted as Chopin. Chopin is predicted frequently because his piano style has highly distinctive surface patterns—dense ornamentation, arpeggios, wide pitch movement, and Romantic-era textures—that the model may over-associate with “composer-specific” signals. When Beethoven

or Mozart passages contain similarly expressive or piano-textured segments, especially in short windows, the model may mistake those local patterns for Chopin even though the broader piece-level style is different.

Interpretation

The CNN's moderate performance suggests that spatial patterns in piano roll representations capture some composer-specific information, but this representation has fundamental limitations for music classification. Several factors contribute to the CNN's underperformance:

Loss of temporal information. CNNs process the piano roll as a static image, applying translation-invariant filters across both time and pitch dimensions. While this approach successfully captures local harmonic structures and note clusters, it struggles to model the sequential dependencies that characterize musical style (Huang et al., 2019). A chord progression that defines a composer's style appears as scattered spatial features rather than a coherent temporal pattern.

Velocity normalization trade-offs. We normalized MIDI velocities to $[0, 1]$ to ensure consistent input scaling. While this prevents the model from overfitting to loudness variations, it may discard stylistically relevant dynamic information. Beethoven's dramatic dynamic contrasts, for instance, become indistinguishable from Mozart's more restrained dynamics (Chew, 2014).

Segment-level classification. Each 10-second segment was classified independently, ignoring the broader musical context. Composer style often emerges over longer time scales through thematic development, formal structure, and motivic transformation—features that short segments cannot capture (Conklin, 2003).

Class similarity. The substantial confusion between Mozart and Beethoven reflects genuine stylistic overlap. Both composers worked within Classical-era conventions, employing

similar textures (Alberti bass, scalar passages), phrase structures (4- and 8-bar phrases), and harmonic progressions (Rosen, 1997). The CNN's spatial filters cannot reliably distinguish these composers when their local note patterns are nearly identical.

Limitations and Future Improvements

Several limitations constrain the CNN's performance in this study:

1. **Architecture simplicity.** Our two-layer CNN may lack the depth required to learn hierarchical musical features. Deeper architectures with residual connections have shown improved performance on music classification tasks (Won et al., 2020).
2. **Limited hyperparameter tuning.** Due to computational constraints, we did not exhaustively search the hyperparameter space. Filter sizes, layer depths, and regularization strengths were selected based on common practices rather than systematic optimization.
3. **Fixed segment length.** The 10-second segment duration was chosen heuristically. Shorter segments may capture insufficient context, while longer segments reduce training data volume and increase computational cost.
4. **Piano roll resolution.** The 10 fps temporal resolution may be too coarse to capture rapid ornamentations and subtle rhythmic variations that distinguish composers (Yang et al., 2017).
5. **Single-channel input.** We used velocity as a single channel. Multi-channel representations incorporating onset detection, sustain, or harmonic features might provide richer information (Dong et al., 2018).

Several directions could improve CNN performance on composer classification:

1. **Architectural enhancements.** Implementing deeper networks with residual connections (He et al., 2016), or employing architectures specifically designed for spectrogram-like inputs such

- as VGGish (Hershey et al., 2017), may improve feature learning. Attention mechanisms could help the model focus on musically salient regions of the piano roll (Huang et al., 2019).
2. Multi-resolution input. Providing the CNN with piano rolls at multiple temporal resolutions (e.g., 5, 10, and 20 fps) could capture both fine-grained note patterns and broader rhythmic structures simultaneously (Schlüter & Böck, 2014).
 3. Data augmentation. Beyond random segment extraction, augmentations such as pitch shifting, time stretching, and velocity perturbation could increase training data diversity and reduce overfitting (McFee et al., 2015).
 4. Hybrid architectures. Combining CNN feature extraction with recurrent layers (CRNN) could preserve spatial pattern recognition while modeling temporal dependencies (Choi et al., 2017). This approach has shown strong results in music tagging and genre classification tasks.
 5. Ensemble methods. Combining predictions from multiple CNN models trained with different initializations or segment lengths could reduce variance and improve overall accuracy (Pons et al., 2018).

Discussion

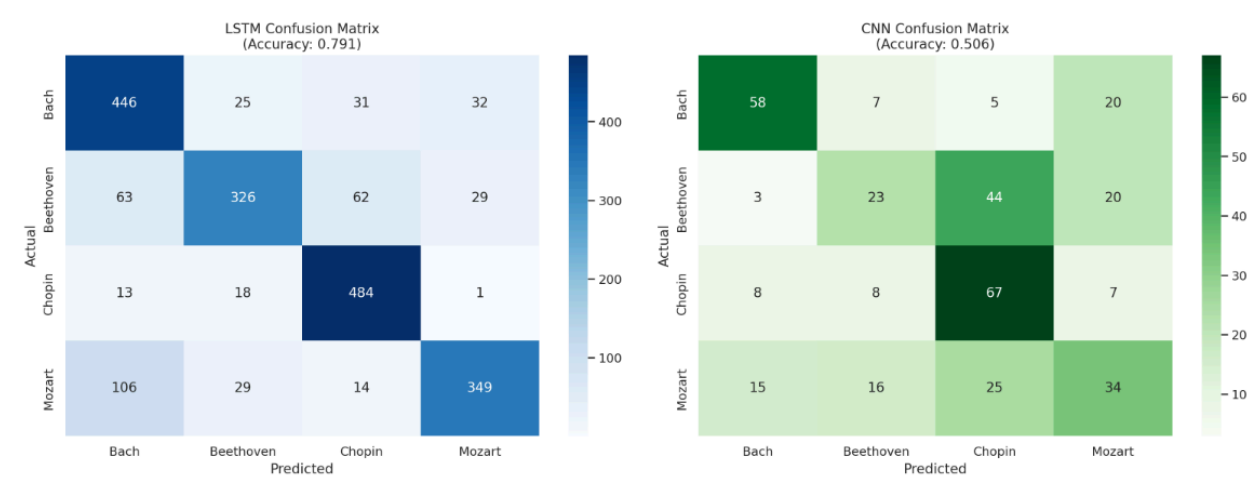
The LSTM significantly outperformed the CNN by about 28 percentage points. This gap was consistent across all four composers, though most pronounced for Mozart and Beethoven.

Table N
Side-by-Side Comparison of LSTM and CNN Model Metrics

Metric	LSTM	CNN

Test Accuracy	79%	51%
Training Epochs	30	~20 (early stopped)

Table 8
Side-by-Side Comparison of LSTM and CNN Model Metrics



The difference is consistent with the information each model receives. The LSTM processes music as a sequence of notes and chords, enabling it to learn ordered relationships such as chord progressions and recurring melodic patterns. This representation aligns with the temporal structure of music, where the order of musical events carries meaning (Hochreiter & Schmidhuber, 1997).

The CNN, by contrast, treats the piano roll as an image. This allows the model to detect spatial patterns such as clustered notes or wide spacing between hands, but it does not directly model how one musical event follows another. As a result, the piano roll representation may lose temporal information that is central to composer style (Choi et al., 2017).

Comparison by Composer

Table 9

Per-Composer F1 Scores for LSTM and CNN Models

Composer	LSTM F1	CNN F1
Bach	0.77	0.67
Beethoven	0.74	0.32
Chopin	0.87	0.58
Mozart	0.77	0.40

Chopin was the easiest to classify for both models. His Romantic piano style—with rolling arpeggios, rubato, and distinctive left-hand patterns—stands out from the three Classical/Baroque composers (Burkholder et al., 2014). Even the CNN could pick up on these spatial patterns.

Bach was the second easiest, likely because Baroque polyphony (multiple independent melodic lines) creates a unique texture that both models can recognize.

Mozart and Beethoven caused the most confusion, especially for the CNN. Both worked in the Classical period and share similar harmonic language, phrase structures, and piano textures (Rosen, 1997). Beethoven even studied Mozart's work extensively (Kinderman, 2009). The LSTM handled this better because it could learn subtle differences in how each composer sequences their musical ideas—something the CNN simply can't see.

Why LSTM Works Better

Our results support what other researchers have found: music is fundamentally sequential, and models that respect this structure perform better (Huang et al., 2019; Deepaisarn et al., 2023). The LSTM's gating mechanism lets it remember important information over long sequences while forgetting irrelevant details (Hochreiter & Schmidhuber, 1997). For composer classification, this means it can track things like "we're in a Bach-style fugue" across many notes, even when individual passages might look similar to other composers. The CNN's translation-invariant filters are great for images where a cat is a cat regardless of where it appears. But in music, *when* something happens matters. A V-I cadence at the end of a phrase means something different than the same notes in the middle. The CNN can't make this distinction (LeCun et al., 1998). That said, CNNs aren't useless for music. Hybrid approaches that combine CNN feature extraction with recurrent layers (CRNNs) have shown strong results in other music tasks (Choi et al., 2017). Our CNN also performed reasonably well on Bach and Chopin, suggesting it does capture *some* useful spatial information.

Conclusion

We compared two deep learning approaches for classifying piano compositions by composer: an LSTM trained on note sequences and a CNN trained on piano roll images. Using MIDI files from Bach, Beethoven, Chopin, and Mozart, we found that the LSTM achieved 79% accuracy while the CNN achieved 51% accuracy.

The findings indicate that sequential representations are more effective for this symbolic music classification task. The LSTM's ability to learn temporal dependencies, including note order and phrase development, proved more effective than the CNN's spatial pattern recognition approach.

For practical applications in music information retrieval, these results suggest that sequence-based models are well suited for symbolic music classification tasks. Researchers working with MIDI data may benefit from representing music as ordered tokens rather than relying exclusively on two-dimensional image-like representations.

This work also highlights the challenge of distinguishing composers with overlapping stylistic characteristics. Mozart and Beethoven were frequently confused by both models, reflecting their shared Classical-era musical vocabulary. This finding suggests that style classification may be a useful complementary direction for future research.

Limitations

1. Dataset size. We used only 90 files per composer after balancing. More data would likely improve both models, especially the CNN which showed signs of overfitting. Larger datasets like the MAESTRO corpus could provide more robust results.
2. Limited composers. Four composers isn't enough to make broad claims about "composer classification" in general. Our findings might not hold for composers with more similar styles (e.g., Haydn vs. Mozart) or more distinct styles (e.g., Debussy vs. Bach).
3. Piano only. All pieces were solo piano works. Orchestral, chamber, or vocal music would present different challenges and might favor different model architectures.
4. MIDI limitations. MIDI captures pitch and timing but loses expressive nuances like articulation, pedaling, and subtle tempo variations (Raffel & Ellis, 2014). These performance details might carry composer-specific information that our models never saw.

5. No hyperparameter optimization. We used reasonable default architectures but didn't systematically tune hyperparameters. The CNN in particular might benefit from deeper architectures or different filter configurations (Won et al., 2020).
6. Single train/test split. We used one 80/20 split. Cross-validation would provide more reliable performance estimates and confidence intervals.
7. No hybrid models. We compared pure LSTM and pure CNN architectures. Hybrid approaches like CRNNs, which combine convolutional feature extraction with recurrent sequence modeling, might outperform both (Choi et al., 2017). Transformer-based models have also shown promise for music tasks (Huang et al., 2019).
8. Segment-level evaluation. We classified individual segments rather than full pieces. A voting or aggregation scheme across segments from the same piece might improve accuracy, especially for the CNN.

References

- Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., Corrado, G. S., Davis, A., Dean, J., Devin, M., Ghemawat, S., Goodfellow, I., Harp, A., Irving, G., Isard, M., Jia, Y., Jozefowicz, R., Kaiser, L., Kudlur, M., ... Zheng, X. (2016). *TensorFlow: Large-scale machine learning on heterogeneous systems*. <https://www.tensorflow.org/>
- Bengio, Y., Simard, P., & Frasconi, P. (1994). Learning long-term dependencies with gradient descent is difficult. *IEEE Transactions on Neural Networks*, 5(2), 157–166. <https://doi.org/10.1109/72.279181>
- Bhandari, K., Roy, A., Wang, K., Puri, G., Colton, S., & Herremans, D. (2024). *Text2midi: Generating symbolic music from captions*. arXiv. <https://arxiv.org/pdf/2412.16526>

- Burkholder, J. P., Grout, D. J., & Palisca, C. V. (2014). *A history of Western music* (9th ed.). W. W. Norton.
- Chew, E. (2014). *Mathematical and computational modeling of tonality: Theory and applications*. Springer. <https://doi.org/10.1007/978-1-4614-9475-1>
- Choi, K., Fazekas, G., Sandler, M., & Cho, K. (2017). Convolutional recurrent neural networks for music classification. *2017 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2392–2396. <https://doi.org/10.1109/ICASSP.2017.7952585>
- Conklin, D. (2003). Music generation from statistical models. *Proceedings of the AISB Symposium on Artificial Intelligence and Creativity in the Arts and Sciences*, 30–35.
- Cuthbert, M. S., & Ariza, C. (2010). music21: A toolkit for computer-aided musicology and symbolic music data. In *Proceedings of the 11th International Society for Music Information Retrieval Conference (ISMIR)* (pp. 637–642).
- Deepaisarn, S., Chokphantavee, S., Chokphantavee, S., Prathipasen, P., Buaruk, S., & Sornlertlamvanich, V. (2023). *NLP-based music processing for composer classification*. *Scientific Reports*, 13, 13228. <https://doi.org/10.1038/s41598-023-40332-0>
- Dong, H. W., Hsiao, W. Y., Yang, L. C., & Yang, Y. H. (2018). MuseGAN: Multi-track sequential generative adversarial networks for symbolic music generation and accompaniment. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), 34–41. <https://doi.org/10.1609/aaai.v32i1.11312>
- Harris, C. R., Millman, K. J., van der Walt, S. J., et al. (2020). Array Programming with NumPy. *Nature* 585, 357 (2020). <https://www.nature.com/articles/s41586-020-2649-2>

- He, K., Zhang, X., Ren, S., & Sun, J. (2016). Deep residual learning for image recognition. 2016 *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 770–778.
<https://doi.org/10.1109/CVPR.2016.90>
- Hershey, S., Chaudhuri, S., Ellis, D. P. W., Gemmeke, J. F., Jansen, A., Moore, R. C., Plakal, M., Platt, D., Saurous, R. A., Seybold, B., Slaney, M., Weiss, R. J., & Wilson, K. (2017). CNN architectures for large-scale audio classification. 2017 *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 131–135.
<https://doi.org/10.1109/ICASSP.2017.7952132>
- Hochreiter, S., & Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735–1780. <https://doi.org/10.1162/neco.1997.9.8.1735>
- Huang, C. Z. A., Vaswani, A., Uszkoreit, J., Simon, I., Hawthorne, C., Shazeer, N., Dai, A. M., Hoffman, M. D., Dinculescu, M., & Eck, D. (2019). Music transformer: Generating music with long-term structure. *Proceedings of the International Conference on Learning Representations (ICLR)*. <https://openreview.net/forum?id=rJe4ShAcF7>
- Le, D.-V.-T., Bigo, L., Herremans, D., & Keller, M. (2025). *Natural language processing methods for symbolic music generation and information retrieval: A survey*. *ACM Computing Surveys*, 57(7), Article 175. <https://doi.org/10.1145/3714457>
- LeCun, Y., Bottou, L., Bengio, Y., & Haffner, P. (1998). Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11), 2278–2324.
<https://doi.org/10.1109/5.726791>
- Kinderman, W. (2009). *Beethoven* (2nd ed.). Oxford University Press.

- McFee, B., Humphrey, E. J., & Bello, J. P. (2015). A software framework for musical data augmentation. *International Society for Music Information Retrieval Conference (ISMIR)*, 248–254.
- Pons, J., Nieto, O., Prockup, M., Schmidt, E., Ehmann, A., & Serra, X. (2018). End-to-end learning for music audio tagging at scale. *International Society for Music Information Retrieval Conference (ISMIR)*.
- Raffel, C., & Ellis, D. P. W. (2014). *Intuitive analysis, creation and manipulation of MIDI data with pretty_midi*. In *Proceedings of the 15th International Society for Music Information Retrieval Conference (ISMIR) Late Breaking and Demo Papers*.
- Rosen, C. (1997). *The Classical Style: Haydn, Mozart, Beethoven* (Expanded ed.). W. W. Norton.
- Schlüter, J., & Böck, S. (2014). Improved musical onset detection with convolutional neural networks. *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 6979–6983.
- Shin, A., & Kaneko, K. (2025). Large language models' internal perception of symbolic music. arXiv. <https://arxiv.org/pdf/2507.12808>
- Won, M., Ferraro, A., Bogdanov, D., & Serra, X. (2020). Evaluation of CNN-based automatic music tagging models. *Sound and Music Computing Conference (SMC)*.
- Wu, S., Guo, Z., Yuan, R., Jiang, J., Doh, S., Xia, G., Nam, J., Li, X., Yu, F., & Sun, M. (2025). CLaMP 3: Universal music information retrieval across unaligned modalities and unseen languages. arXiv. <https://arxiv.org/pdf/2502.10362>

Yang, D., Ji, K., & Tsai, T. (2021). A deeper look at sheet music composer classification using self-supervised pretraining. *Applied Sciences*, 11(4), 1387.

<https://doi.org/10.3390/app11041387>

Xu, W., McAuley, J., Berg-Kirkpatrick, T., Dubnov, S., & Dong, H.-W. (2024). Generating symbolic music from natural language prompts using an LLM-enhanced dataset. *arXiv*.

<https://arxiv.org/pdf/2410.02084>